

ROS2 Spring Workshop

Supplemental Guide: Getting Your WSL2 Dev Container Talking to the Robot

Who This Is For

You have followed the lab. Your robot is powered on, you are on the right Wi-Fi, your `ROS_DOMAIN_ID` is set, and `ros2 topic list` returns nothing but `/parameter_events` and `/rosout`. Meanwhile, your classmate on a native Linux laptop sees the full list. **This guide fixes that, specifically for Labs 8, 9, and 10.**

The root cause: WSL2's default networking quietly drops the multicast packets that ROS 2's DDS layer uses to discover nodes on the network. Docker Desktop adds a second layer of the same problem. We fix both, in order, so the dev container at `/workspaces/agtech_ros2` can see the robot's topics across Wi-Fi.

The Plan

Main path (Parts 0–5): Fix WSL networking, then configure Docker Desktop and the dev container so DDS multicast actually reaches your code. This is what 95% of you will use.

Plan B (Part 6): If the dev container keeps fighting you, install ROS Humble directly inside WSL and move your lab code over. We show you exactly how. You lose the prebuilt environment but gain networking that just works.

Either way, you'll finish Labs 8, 9, and 10.

Pre-Fix Verification (90 Seconds, Do This First)

If any of these fail, the rest of the guide silently won't work. Check them before touching anything.

Check 1: Windows version

```
1 winver
```

You need **Windows 11, version 23H2 or newer**. The Hyper-V firewall cmdlet we use later was added in 23H2.

Check 2: WSL version

```
1 wsl --version
```

You need WSL 2.0.0 or newer for mirrored networking. If older, run `wsl -update`.

Check 3: Wi-Fi network profile is Private

Settings → Network & Internet → Wi-Fi → click your network name → verify “Network profile type” is **Private**. **Campus and dorm Wi-Fi often auto-flag as Public, which causes Windows Defender Firewall to block inbound traffic regardless of everything else.**

Check 4: No stale ROS environment variables

In a WSL terminal:

```
1 echo "LOCALHOST_ONLY: $ROS_LOCALHOST_ONLY"
2 unset ROS_LOCALHOST_ONLY
```

If `ROS_LOCALHOST_ONLY` was set to anything (even 0), it blocks multi-machine discovery. Unset it.

Pre-Fix Concept Primer

1. WSL2 Networking Modes (NAT vs. Mirrored)

By default, WSL2 puts your Linux distribution behind a **NAT**. Your WSL gets a private IP like 172.x.x.x that no other device on the LAN can reach. Fine for browsing, broken for ROS 2.

Mirrored mode replaces NAT with a stack that “mirrors” your Windows network interfaces directly into Linux. WSL gets the same LAN IP as Windows, and multicast actually works.

2. DDS Multicast Discovery

ROS 2 nodes use **UDP multicast** to broadcast “hello, I exist” messages. If any layer between your code and the network blocks multicast, nodes never find each other, even though SSH and ping work fine. The whole point of this guide is making sure multicast reaches your dev container.

3. RMW Implementations

ROS 2 supports multiple DDS vendors. **Our setup uses Cyclone DDS (rmw_cyclonedds_cpp)**. The robot’s container uses Cyclone, the instructor’s God Node uses Cyclone, and your dev container needs to use Cyclone too. Two machines using different RMWs cannot discover each other, even if multicast works.

4. The Hyper-V Firewall (Yes On Home Edition Too)

Even though Hyper-V the product is missing on Windows 11 Home, the underlying **Hyper-V networking stack** is present. WSL2 runs on it, and it includes its own firewall layer separate from Windows Defender. By default it blocks inbound traffic to WSL. You won’t see it in the normal Windows Security GUI; we configure it through PowerShell.

5. Docker Desktop’s Host Networking

A Docker container normally sits behind a virtual bridge: even if WSL networking is fixed, the container is still NAT’d from the LAN. **Host networking** (a Docker Desktop feature) tells the container to skip the bridge and share the host’s network stack directly. Combined with WSL mirrored mode, the container ends up speaking directly to the LAN. This is what lets multicast reach your dev container.

Part 0: Prerequisites (Windows 11 Home Users)

Skip This If...

You are on Windows 11 **Pro** or **Enterprise**. The features below are enabled by default there. Home users, do not skip.

Start → “Turn Windows features on or off” → verify all three are checked:

- Virtual Machine Platform
- Windows Subsystem for Linux
- Windows Hypervisor Platform

“**Hyper-V**” itself will be greyed out or simply missing on Home. That is expected. You don’t need it; you need the three above.

If you had to enable any of these, **reboot before continuing**.

Part 1: Edit .wslconfig

WSL reads a global config file at startup. We use it to turn on mirrored networking and relax the Hyper-V firewall.

1. Open **PowerShell** (regular, not admin).
2. Open or create the config in Notepad:

```
1 notepad $env:USERPROFILE\.wslconfig
2
```

3. Paste exactly this content:

```
1 [wsl2]
2 networkingMode=mirrored
3 firewall=false
4 localhostForwarding=true
5 dnsTunneling=true
6
7 [experimental]
8 hostAddressLoopback=true
9
```

4. Save and close.

Decoding the Config Keys

- **networkingMode=mirrored**: WSL shares the Windows host's network interfaces directly. This is the core fix.
- **firewall=false**: Disables the Hyper-V firewall layer for WSL traffic. Windows Defender still protects the host.
- **localhostForwarding=true**: Lets 127.0.0.1 cross between Windows and WSL. Standard hygiene.
- **dnsTunneling=true**: Routes DNS through Windows. Helps on campus/corporate networks where the WSL resolver gets confused under mirrored mode.
- **hostAddressLoopback=true**: Lets the Windows host be reached from WSL by its LAN IP. Docker Desktop relies on this.

Part 2: Allow Inbound on the WSL VM

Even with `firewall=false` in `.wslconfig`, set the Hyper-V firewall default to “Allow” as a backup.

1. **PowerShell as Administrator** (right-click → Run as administrator).
2. Run:

```
1 Set-NetFirewallHyperVVMSetting -Name '{40E0AC32-46A5-438A-A0B2-2B479E8F2E90}' -
   DefaultInboundAction Allow
2
```

3. No output means success. An error means Part 0 isn't done, or you're on a Windows version older than 23H2.

The GUID is a fixed identifier Microsoft assigns to the WSL VM. It's the same on every Windows install.

Part 3: Restart WSL (Don't Skip This)

The `.wslconfig` file is only read when the WSL VM starts. Closing your terminal is not enough; the VM keeps running.

1. Close every WSL terminal window.

2. In PowerShell:

```
1 wsl --shutdown
2
```

3. Wait 10 seconds.

4. Restart Docker Desktop from the system tray. (Docker Desktop runs its own WSL distro and needs to pick up the new networking too.)

5. Open a fresh WSL terminal.

The 1 Mistake

Students edit `.wslconfig`, skip `wsl -shutdown`, then spend an hour wondering why nothing changed. Always shut down WSL after editing the config.

Part 4: Quick Sanity Check (Before Touching Docker)

Before fighting Docker, prove the WSL layer is working. In your fresh WSL terminal:

```
1 ip addr show
```

You should see an interface with the same IP as your Windows machine (check with `ipconfig` from PowerShell). If WSL still has a `172.x.x.x` address, mirrored mode didn't take effect: go back and verify the config and the shutdown.

If you have ROS installed in WSL directly, also run:

```
1 ros2 multicast receive
```

And from the robot or another Linux machine on the same network:

```
1 ros2 multicast send
```

If WSL receives the "Hello World," the WSL networking layer is good. Now make Docker pass it through.

Part 5: Make the Dev Container Work

You need three things lined up at once: Docker Desktop's host networking enabled, `-network=host` added to your dev container, and Cyclone DDS configured inside.

Step 1: Enable Docker Desktop Host Networking

Docker Desktop → **Settings** → **Resources** → **Network** → check "Enable host networking." Click Apply & Restart.

Without this toggle, `-network=host` is silently accepted and ignored. The container still gets a bridge IP, and you'll spend an hour debugging something that isn't actually configured.

Step 2: Edit `devcontainer.json`

Open `.devcontainer/devcontainer.json` in your repo and add `runArgs` plus a `remoteEnv` block:

```
1 {
2   "name": "agtech_ros2",
3   "image": "your-existing-image",
4   "runArgs": ["--network=host"],
5   "remoteEnv": {
6     "RMW_IMPLEMENTATION": "rmw_cyclonedds_cpp"
7   }
8 }
```

Change `ROS_DOMAIN_ID` to your group's ID. Remove any `forwardPorts` or `ports` entries; they conflict with host networking and will warn.

Step 3: Rebuild the Container

In VS Code: Command Palette (Ctrl+Shift+P) → **Dev Containers: Rebuild Container**. The new run args only apply on a fresh container build, not just a reopen.

Step 4: Install Lab Dependencies Inside the Container

Lab 10 needs pynput for the keyboard deadman:

```
1 pip install pynput
2 # If pip complains: pip install pynput --break-system-packages
```

Make sure Cyclone is installed (it usually is, but verify):

```
1 sudo apt update
2 sudo apt install -y ros-humble-rmw-cyclonedds-cpp
```

Step 5: Verify

Inside the dev container, in a fresh terminal:

```
1 echo $ROS_DOMAIN_ID
2 echo $RMW_IMPLEMENTATION
3 # both should show what you set in devcontainer.json
4
5 pkill -f ros2_daemon
6 ros2 topic list
```

If ros2 daemon stop Hangs

On WSL, `ros2 daemon stop` can hang indefinitely after a networking change because the daemon is stuck waiting on a socket that no longer exists. Confirmed in the field, not hypothetical. Use `pkill -f ros2_daemon` instead. The next ROS 2 command spawns a fresh daemon automatically.

You should see the robot's topics: `/cmd_vel`, `/odom`, `/tractor/gps` (when the God Node is running), and a long list of others. If you do, you're done. Skip Plan B and go work on your lab.

If topics are still missing, run the multicast test:

```
1 # Inside dev container:
2 ros2 multicast receive
3
4 # On robot or another Linux machine:
5 ros2 multicast send
```

If the container sees nothing, host networking isn't actually on. Re-check Step 1 (it sometimes gets disabled by Docker Desktop updates) and verify the container was rebuilt, not just restarted.

Part 6: Plan B – Native WSL Install

If Part 5 keeps fighting you, install ROS Humble directly inside WSL Ubuntu, move your lab code over, and skip Docker entirely. You lose the dev container's prebuilt environment, but networking just works.

Trade-offs:

- You diverge from the course's standard container. If the instructor adds packages later, you install them yourself.
- Builds run on the native Linux filesystem: 5–20× faster than the dev container.
- Networking just works. No Docker Desktop, no host networking toggle.

Step 1: Install Ubuntu in WSL If You Haven't (You Have Tho...)

```
1 wsl --install -d Ubuntu-22.04
```

ROS 2 Humble targets Ubuntu 22.04 specifically. Don't use 24.04.

Step 2: Install ROS 2 Humble

```
1 # Check locale first, if this shows UTF-8 you can skip the locale block entirely
2 locale
3
4 # Add ROS repo
5 sudo apt update
6 sudo apt install -y curl gnupg
7 sudo add-apt-repository universe
8 sudo curl -sSL https://raw.githubusercontent.com/ros/rosdistro/master/ros.key \
9 -o /usr/share/keyrings/ros-archive-keyring.gpg
10 echo "deb [arch=$(dpkg --print-architecture) signed-by=/usr/share/keyrings/ros-archive-
    keyring.gpg] http://packages.ros.org/ros2/ubuntu $(. /etc/os-release && echo
    $UBUNTU_CODENAME) main" \
11 | sudo tee /etc/apt/sources.list.d/ros2.list > /dev/null
12
13 # Install
14 sudo apt update
15 sudo apt install -y ros-humble-desktop python3-colcon-common-extensions \
16 python3-pip ros-humble-rmw-cyclonedds-cpp
17
18 # Make every new shell ROS-aware and Cyclone-aware
19 echo 'source /opt/ros/humble/setup.bash' >> ~/.bashrc
20 echo 'export RMW_IMPLEMENTATION=rmw_cyclonedds_cpp' >> ~/.bashrc
21 source ~/.bashrc
```

Step 3: Install Lab Dependencies

```
1 pip install pynput numpy matplotlib
2 # pynput is for Lab 10's keyboard deadman.
```

Step 4: Move Your Code Into WSL

The Filesystem Geography

From WSL, Windows files are at: /mnt/c/Users/<you>/...

From Windows, WSL files are at: \\wsl.localhost\Ubuntu-22.04\home\<you>\ (paste into File Explorer).

Rule: always keep your code in the WSL native filesystem (~/ or anywhere under /home). Code on /mnt/c/ is 5–20× slower to build because every file operation crosses the Windows–Linux boundary.

If your code is on Git (best case):

```
1 cd ~
2 mkdir -p ros2_ws/src
3 cd ros2_ws/src
4 git clone <the_repo_url> agtech_ros2
```

If your code is in the old dev container: open the container one last time, push to Git or copy out:

```
1 # In the dev container:
2 cp -r /workspaces/agtech_ros2 /mnt/c/Users/<you>/Desktop/agtech_backup
3
4 # Then from WSL (not the container):
5 mkdir -p ~/ros2_ws/src
6 cp -r /mnt/c/Users/<you>/Desktop/agtech_backup ~/ros2_ws/src/agtech_ros2
```

Step 5: Recreate the Lab's Path Convention

The lab manuals refer to /workspaces/agtech_ros2. Symlink it so the same paths work:

```
1 sudo mkdir -p /workspaces
2 sudo ln -s ~/ros2_ws/src/agtech_ros2 /workspaces/agtech_ros2
```

Step 6: Build

```
1 cd ~/ros2_ws
2 colcon build --symlink-install
3 echo 'source ~/ros2_ws/install/setup.bash' >> ~/.bashrc
4 source ~/.bashrc
```

-symlink-install lets you edit Python files without rebuilding every time. The labs benefit from this.

Step 7: Use VS Code with Remote-WSL

You edit code from Windows VS Code via the Remote-WSL extension. No file shuffling.

1. Install the **WSL** extension in VS Code (Microsoft, not the Dev Containers one).
2. In WSL terminal: `code ~/ros2_ws`. VS Code opens with “[WSL: Ubuntu-22.04]” in the title bar. That’s how you know it’s editing inside WSL.
3. Terminals opened in that window are WSL terminals. Run `colcon build`, `ros2 run`, etc. directly.

Step 8: Verify

```
1 pkill -f ros2_daemon
2 ros2 topic list
```

Topics should appear. You’re done.